

派大海星

首页

新随笔

管理

Keywords

libusb 批传输的使用方法

概述

libusb是开发与USB外围设备通讯应用程序的好帮手，其API精益且功能强大，完备内容可以查看接口文档和库提供的 **examples** 示例程序。

自己使用较多的是USB批量传输类型，在此记录下有关批量传输通讯需要的一些步骤。代码适用于存在批量传输端点的USB设备。

数据结构

1. 查找设备的标识

```
struct bulkusbid
{
    unsigned short vid;
    unsigned short pid;
};
```

2. 一些必须参数

```
struct bulkusbdev
{
    libusb_device_handle* handle;
    uint8_t endpoint_in;
    uint8_t endpoint_out;
    uint8_t bus;
    uint8_t addr;
    int nb_ifaces;
};
```

初始化和退出

```
int bulkusb_init()
{
    return libusb_init(NULL);
}
```

```
void bulkusb_exit()
{
    libusb_exit(NULL);
}
```

查找设备

同一产品多个设备它们的pid、vid都是一样的，这里获取这个id下的所有设备以备选择。

```
int bulkusb_get_list(struct bulkusbdev dev_list[], int dev_max_cnt, struct bulkusbid id_list[], unsigned int id_cnt)
{
    int dev_cnt = 0;
    struct libusb_device_descriptor desc;
    libusb_device** devs;
    ssize_t cnt;
    int r, i;
    unsigned int j;

    cnt = libusb_get_device_list(NULL, &devs);
    if (cnt < 0)
        return (int)cnt;

    for (i = 0; devs[i]; ++i)
    {
        if (dev_cnt >= dev_max_cnt)
        {
            break;
        }

        r = libusb_get_device_descriptor(devs[i], &desc);
        if (r < 0)
        {
            fprintf(stderr, "failed to get device descriptor");
            return r;
        }

        for (j = 0; j < id_cnt; j++)
        {
            if (id_list[j].vid == desc.idVendor &&
                id_list[j].pid == desc.idProduct)
            {
                dev_list[dev_cnt].bus = libusb_get_bus_number(devs[i]);
                dev_list[dev_cnt].addr = libusb_get_device_address(devs[i]);
                dev_cnt++;
            }
        }
    }

    libusb_free_device_list(devs, 1);

    return dev_cnt;
}
```

打开设备

这部分代码主要有以下作用：

1. 获取批量读写所需的必须参数 句柄 、 输入端点 、 输出端点 ；
2. 清除端点的暂停标志并重新；
3. 声明接口，这些接口关闭时要释放；
4. 重新初始化设备；

这里获取端点需要注意的几点是：

1. 只查找第一个设备配置描述符；
2. 只查找厂商自定义类型接口或者CDC类型接口；
3. 只使用找到的第一个输入输出端点；
4. 如果已知输入输出端点，也可以直接写定它们的值，不使用查找的方法；

```
int bulkusb_open(struct bulkusbdev* bdev)
{
    libusb_device** devs;
    ssize_t cnt;
    int r, i;

    cnt = libusb_get_device_list(NULL, &devs);
    if (cnt < 0)
    {
        printf("\n\nlibusb error: %s\n", libusb_strerror((enum libusb_error)cnt));
        return (int)cnt;
    }

    libusb_device* dev = NULL;
    libusb_device_handle* handle = NULL;

    for (i = 0; devs[i]; ++i)
    {
        dev = devs[i];
        if (bdev->bus == libusb_get_bus_number(devs[i])
            && bdev->addr == libusb_get_device_address(devs[i]))
        {
            //获得句柄
            if (1)
            {
                r = libusb_open(dev, &handle);
                if (LIBUSB_SUCCESS != r)
                {
                    libusb_free_device_list(devs, 1);
                    printf("\n\nlibusb error: %s\n", libusb_strerror((enum libusb_error)r));
                    return r;
                }
                bdev->handle = handle; //赋值
            }

            //获得通讯端点并声明内核接口
            if (1)
            {
                int i, j, k;
```

```

struct libusb_config_descriptor* conf_desc;
const struct libusb_endpoint_descriptor* endpoint;
uint8_t endpoint_in = 0, endpoint_out = 0; // default IN and OUT endpoints
int iface, nb_ifaces;
uint8_t interfaceClass;

libusb_get_config_descriptor(dev, 0, &conf_desc); //uint8_t config_index = 0;
nb_ifaces = conf_desc->bNumInterfaces;
bdev->nb_ifaces = nb_ifaces; //赋值
for (i = 0; i < nb_ifaces; i++) {
    for (j = 0; j < conf_desc->interface[i].num_altsetting; j++) {
        //只获取接口类别为：厂商自定义类 (0xFF) 和CDC数据类 (0xA)
        interfaceClass = conf_desc->interface[i].altsetting[j].bInterfaceClass;
        if (interfaceClass != 0xFF && interfaceClass != 0x0A) {
            continue;
        }
        for (k = 0; k < conf_desc->interface[i].altsetting[j].bNumEndpoints; k++)
            struct libusb_ss_endpoint_companion_descriptor* ep_comp = NULL;
        endpoint = &conf_desc->interface[i].altsetting[j].endpoint[k];
        // Use the first bulk IN/OUT endpoints as default for testing
        if ((endpoint->bmAttributes & LIBUSB_TRANSFER_TYPE_MASK) & (LIBUSB_TRANSFER_TYPE_BULK & LIBUSB_ENDPOINT_IN)) {
            if (endpoint->bEndpointAddress & LIBUSB_ENDPOINT_IN) {
                if (!endpoint_in)
                {
                    endpoint_in = endpoint->bEndpointAddress;
                    bdev->endpoint_in = endpoint_in; //赋值
                    libusb_clear_halt(handle, bdev->endpoint_in); //清除暂停标志
                }
            }
            else {
                if (!endpoint_out)
                {
                    endpoint_out = endpoint->bEndpointAddress;
                    bdev->endpoint_out = endpoint_out; //赋值
                    libusb_clear_halt(handle, bdev->endpoint_out);
                }
            }
        }
    }
}

libusb_free_config_descriptor(conf_desc);

libusb_set_auto_detach_kernel_driver(handle, 1);
for (iface = 0; iface < nb_ifaces; iface++)
{
    r = libusb_claim_interface(handle, iface);
    if (r != LIBUSB_SUCCESS) {
        printf("    Failed.\n");
    }
}

}

libusb_free_device_list(devs, 1);

```

```

if (bdev->endpoint_in == 0 || bdev->endpoint_out == 0)
{
    printf("*    Failed:endpoint_in or endpoint_out is NULL!\n");
    return -99;
}
//重新初始化
libusb_reset_device(bdev->handle);
return 0;
}

```

关闭设备

```

int bulkusb_close(struct bulkusbdev* bdev)
{
    if (bdev->handle)
    {
        int iface;
        for (iface = 0; iface < bdev->nb_ifaces; iface++) {
            printf("Releasing interface %d...\n", iface);
            libusb_release_interface(bdev->handle, iface);
        }

        libusb_close(bdev->handle);
    }

    return 0;
}

```

批量写

```

int bulkusb_write(struct bulkusbdev* dev, void* buffer, int len, int ms)
{
    int size, errcode;
    libusb_device_handle* handle = dev->handle;
    uint8_t endpoint_out = dev->endpoint_out;

    errcode = libusb_bulk_transfer(handle, endpoint_out, buffer, len, &size, ms);
    if (errcode < 0)
    {
        printf("write:    %s\n", libusb_strerror((enum libusb_error)errcode));
        return -1;
    }

    return size;
}

```

批量读

```

int bulkusb_read(struct bulkusbdev* dev, void* buffer, size_t len, int ms)
{

```

```

int size, errcode;
libusb_device_handle* handle = dev->handle;
uint8_t endpoint_in = dev->endpoint_in;

errcode = libusb_bulk_transfer(handle, endpoint_in, buffer, len, &size, ms);
if (errcode < 0)
{
    printf("read:   %s\n", libusb_strerror((enum libusb_error)errcode));
    return -1;
}

return size;
}

```

步骤执行顺序

```

char buf[8] = { 0 };
struct bulkusbdev* dev = NULL;
struct bulkusbdev devs[2] = { 0 };
struct bulkusbid ids[1] = {
    {
        .vid = 0x1234,
        .pid = 0x1122
    }
};

bulkusb_init();
bulkusb_get_list(devs, 2, ids, 1);
dev = &devs[0];
bulkusb_open(dev);
bulkusb_write(dev, buf, 8, 1000);
bulkusb_read(dev, buf, 8, 1000);
bulkusb_close(dev);
bulkusb_exit();

```

分类: C/C++

好文要顶

关注我

收藏该文

微信分享



派大海星

粉丝 - 5 关注 - 2

+加关注

« 上一篇: Qt 数据库操作

» 下一篇: Qt UI界面美化

posted @ 2020-08-20 08:56 派大海星 阅读(2337) 评论(0) 编辑 收藏 举报

0

0

刷新页面 返回顶部

登录后才能查看或发表评论，立即 [登录](#) 或者 [逛逛](#) 博客园首页

- 【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
- 【推荐】还在用 ECharts 开发大屏？试试这款永久免费的开源 BI 工具！
- 【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
- 【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
- 【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步



- 主页
- 新随笔
- 分类
- 概述
- 数据结构
- 初始化和退出
- 查找设备
- 打开设备
- 关闭设备
- 批量写
- 批量读
- 步骤执行顺序
- 公告
- 昵称： 派大海星
- 园龄： 5年5个月
- 粉丝： 5
- 关注： 2
- +加关注
- 我的标签
- tools–vscode(5)
- linux–shell(4)

[tools-markdown\(3\)](#)

[linux-ops\(3\)](#)

[tools-gdb\(2\)](#)

[esp\(2\)](#)

[c/c++-opencv\(2\)](#)

[c/c++-library\(2\)](#)

[tools-gcc\(1\)](#)

[tools-cmake\(1\)](#)

[杂杂的\(1\)](#)

[随笔分类](#) (49)

Copyright © 2025 派大海星

Powered by .NET 9.0 on Kubernetes